

CARRERA DE INGENIERÍA DE SOFTWARE

EXAMEN PRÁCTICO — UNIDAD 1

Generalidades de los Sistemas Distribuidos

Datos generales

Asignatura	Aplicaciones Distribuidas (ISR-701)
Unidad	1 — Generalidades de los Sistemas Distribuidos
Tipo de evaluación	Examen práctico de unidad — Individual
Modalidad	Laboratorio con defensa técnica oral
Duración	2 horas de laboratorio
Período académico	Regular 2025-2026 SPA
Código del instrumento	EV-PR-U1 (ajustable según planificación)
Docente	_____

Estudiante	_____
Fecha	_____
URL del repositorio	_____
Calificación (sobre 10)	_____

1. Competencia y resultado de aprendizaje evaluados

Este examen valora la capacidad del estudiante para analizar las características y principios de los sistemas distribuidos, evaluando sus ventajas, limitaciones y retos desde una perspectiva crítica, y para implementar de forma funcional los mecanismos fundamentales de la unidad: comunicación entre procesos, sincronización mediante relojes lógicos, tolerancia a fallos, coordinación y seguridad básica.

2. Planteamiento del problema

Una empresa de logística opera un servicio de registro de operaciones llamado RegistroDistribuido. El servicio se compone de tres nodos réplica (N1, N2, N3) que reciben “operaciones” (por ejemplo, registros de movimientos) provenientes de uno o varios clientes. Por razones de disponibilidad, los nodos deben coordinarse entre sí, mantener un orden coherente de los eventos que registran y seguir funcionando aunque alguno de ellos deje de responder.

Su tarea es construir un prototipo funcional de este servicio que demuestre, sobre un caso pequeño y ejecutable, los principios estudiados en la Unidad 1. No se evalúa la escala ni la persistencia en disco, sino la correcta aplicación de los mecanismos distribuidos. El sistema puede ejecutarse como tres procesos en la misma máquina (distintos puertos) o como tres contenedores.

3. Requerimientos a implementar

Parte A — Comunicación entre procesos (peso orientativo 20%)

- Implemente la comunicación cliente–nodo (y nodo–nodo) usando sockets TCP o gRPC.

- Delimite correctamente los mensajes (longitud previa, separador o framing) de modo que dos operaciones enviadas seguidas no se solapen ni se trunquen en el receptor.
- Un cliente debe poder enviar una operación y recibir la confirmación del nodo.

Parte B — Relojes lógicos de Lamport (peso orientativo 20%)

- Cada nodo mantiene un reloj lógico de Lamport e implementa correctamente sus reglas: incremento ante un evento local y, al recibir un mensaje, fijar el reloj en $\max(\text{local}, \text{recibido}) + 1$.
- El sistema debe generar un registro (log) de eventos con un orden total consistente, usando el identificador de nodo como criterio de desempate ante marcas iguales.

Parte C — Tolerancia a fallos (peso orientativo 15%)

- Los nodos intercambian latidos (heartbeats) periódicos.
- Si un nodo deja de responder, los demás lo marcan como caído dentro de un tiempo acotado, registran el evento y el sistema continúa operando con los nodos restantes (gestión de fallo parcial).

Parte D — Coordinación (peso orientativo 10%)

- Implemente la elección de coordinador con el algoritmo Bully: el nodo activo de mayor identificador actúa como coordinador.
- Ante la caída del coordinador, el sistema debe disparar y completar una nueva elección sin intervención manual.

Parte E — Seguridad (peso orientativo 10%)

- El cliente debe autenticarse mediante un token antes de que el nodo acepte sus operaciones; las solicitudes sin token válido se rechazan.

4. Componente de análisis y defensa técnica

Al finalizar la implementación, el estudiante registrará sus respuestas en un archivo ANALISIS.md y las defenderá oralmente ante el docente. Las preguntas conectan el prototipo con la teoría de la unidad:

1. Ante una interrupción de comunicación entre dos nodos, ¿qué propiedad del teorema CAP privilegia su implementación y por qué? Justifíquelo con el comportamiento observado en su prototipo.
2. ¿Qué falacias de la computación distribuida tuvo que considerar al delimitar los mensajes y al definir los tiempos de espera de los latidos?
3. ¿Qué tipos de transparencia (ubicación, acceso, fallos, replicación) ofrece o no ofrece su solución? Argumente cada caso.
4. Proponga un acuerdo de nivel de servicio (SLA) de disponibilidad para este sistema y calcule el tiempo de inactividad anual admisible que implicaría.
5. Si reemplazara el algoritmo Bully por un consenso tipo Raft, ¿qué ganaría y qué costo introduciría?

5. Entregables y condiciones

- **Repositorio:** Repositorio Git con el código fuente y un README que indique cómo compilar y ejecutar el prototipo.
- **Análisis:** Archivo ANALISIS.md con las respuestas del componente de análisis.
- **Demostración:** Demostración en vivo: levantar los tres nodos, enviar operaciones, mostrar el orden total de eventos y provocar la caída de un nodo (incluido el coordinador).

- **Condición:** El prototipo debe compilar y ejecutarse siguiendo las instrucciones del README. Un proyecto que no ejecuta no puede acreditar los criterios funcionales.

6. Restricciones técnicas

- Lenguaje y stack: Java 21 + Spring Boot 3.x, o Python 3.11+. Comunicación mediante sockets TCP o gRPC.
- Ejecución local (varios puertos) o en contenedores Docker; IDE de referencia: IntelliJ IDEA.
- El uso de asistentes de IA y de material de consulta se rige por la política indicada por el docente para esta evaluación. En la defensa, el estudiante debe poder explicar y justificar cada parte de su código sin leerlo textualmente.

7. Rúbrica de evaluación

Cada criterio se valora en cuatro niveles: Excelente (4), Bueno (3), Regular (2) y Deficiente (1). Un criterio no entregado, con plagio verificado o que no corresponde recibe 0. La calificación final se obtiene como la suma ponderada de los niveles alcanzados, según la fórmula:

$$\text{Nota (sobre 10)} = [\sum (\text{peso}_i \times \text{nivel}_i) \div 4] \times 10$$

Donde los pesos se expresan como fracción (sumando 1,0) y el nivel es el valor 1 a 4 alcanzado en cada criterio. Si todos los criterios alcanzan el nivel Excelente, la nota es 10.

Rúbrica de evaluación — Examen Práctico Unidad 1

Criterio	Peso	Excelente (4)	Bueno (3)	Regular (2)	Deficiente (1)
C1. Comunicación entre procesos y delimitación de mensajes Peso: 20%	20%	Cliente y nodos se comunican por sockets TCP o gRPC; los mensajes se delimitan correctamente y nunca se solapan ni truncan; maneja varias operaciones seguidas sin pérdida.	La comunicación funciona y delimita los mensajes, con fallos menores ante concurrencia o casos límite.	Funciona en el caso simple pero no delimita los mensajes (se solapan/truncan) o solo atiende a un cliente.	La comunicación es inestable o no se establece de forma fiable.
C2. Relojes lógicos de Lamport y orden total Peso: 20%	20%	Aplica correctamente las reglas (incremento local y max+1 al recibir) y produce un log con orden total consistente, con desempate por ID; el orden respeta la causalidad.	Reglas correctas; el orden total funciona salvo en empates o detalles menores.	Implementa el contador pero aplica mal una regla (p. ej. no toma el máximo al recibir) o no genera orden total.	Relojes ausentes o incorrectos; no hay ordenamiento de eventos.
C3. Tolerancia a fallos: heartbeat y continuidad Peso: 15%	15%	Los nodos intercambian latidos; la caída se detecta en tiempo acotado, se registra y el sistema sigue operando con los nodos restantes.	Detecta la caída y continúa, con detección lenta o registro incompleto.	Detecta la caída pero el sistema se degrada de forma incorrecta o inestable.	No hay detección de fallos; la caída de un nodo detiene el sistema.
C4. Coordinación: elección de líder (Bully) Peso: 10%	10%	Implementa Bully correctamente; elige al nodo activo de mayor ID y, ante la caída del líder, dispara y completa una nueva elección sin intervención manual.	La elección funciona, pero la reelección tras la caída es parcial o requiere ajustes.	Designa un coordinador con criterio fijo, sin elección dinámica real.	No hay coordinación o el coordinador es estático/codificado a mano.
C5. Seguridad: autenticación del cliente Peso: 10%	10%	El nodo rechaza operaciones sin token válido y acepta solo clientes autenticados; el manejo del token es coherente.	Autentica correctamente, con detalles menores en el manejo del token.	Verifica el token de forma superficial o fácilmente evadible.	No hay autenticación; cualquier cliente es aceptado.
C6. Calidad del código, ejecución y repositorio Peso: 10%	10%	Compila/ejecuta siguiendo el README; código organizado y legible; instrucciones claras; repositorio con historial de commits.	Ejecuta con ajustes menores; organización aceptable.	Ejecuta con dificultad o instrucciones incompletas; código desordenado.	No compila/ejecuta o no hay repositorio utilizable.
C7. Análisis y defensa técnica Peso: 15%	15%	Responde con precisión las preguntas, justificando el compromiso CAP, las falacias, transparencias y un SLA razonable; defiende su código sin leerlo y razona ante preguntas inesperadas.	Análisis correcto en lo esencial con alguna imprecisión; defensa sólida.	Análisis superficial o parcialmente erróneo; defensa dubitativa.	No conecta la práctica con la teoría; no defiende su solución.

Nota: “0 puntos” se asigna a un criterio no entregado, con plagio verificado o que no corresponde al criterio. La calificación final aplica la fórmula ponderada indicada en la sección 7.